

OOP — Core Pillars

Let's build the mental model first, then go deep into **Encapsulation vs Abstraction**, because this is a very common interview area.

1. What is OOP?

OOP (Object-Oriented Programming) is a programming paradigm where we organize software around **objects**, rather than primarily around functions and procedures.

An **object** combines:

- **Data** → what it has
- **Methods** → what it can do

For example, a **BankAccount** object:

```
class BankAccount {
  private:
    double balance;

  public:
    void deposit(double amount) {
      balance += amount;
    }

    double getBalance() {
      return balance;
    }
};
```

Here:

- **balance** → data/state
- **deposit()** → behavior
- **BankAccount** → class
- An actual bank account created from it → **object**

Mental model

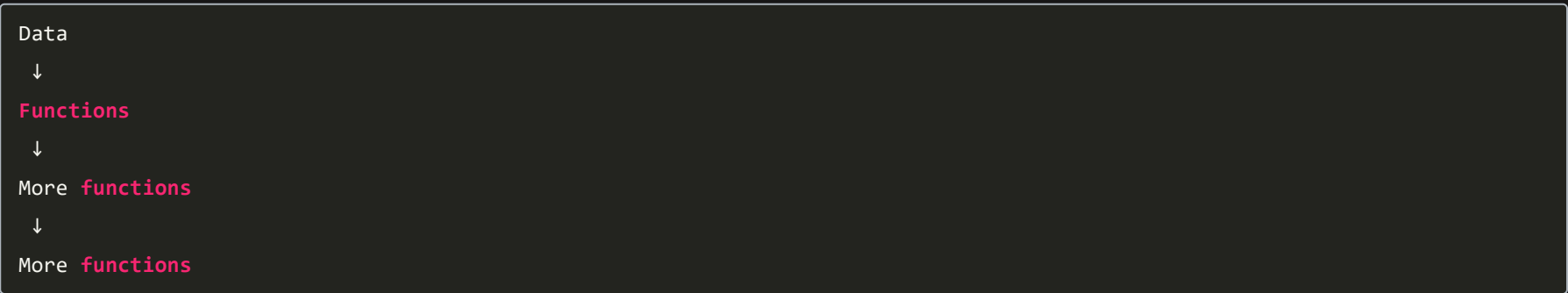
Think:

Object = State + Behavior

2. Why OOP instead of procedural programming?

Procedural programming

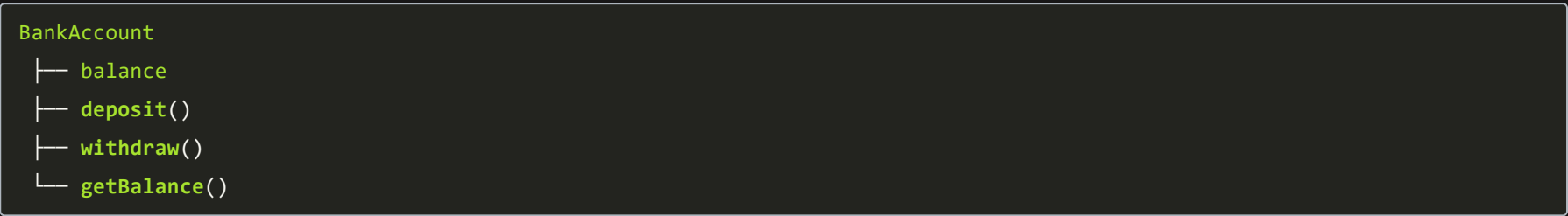
The focus is mainly on **functions/procedures** that operate on data.



As applications become large, it can become difficult to control which functions can modify which data.

OOP

We group related **data + functions** together.



This gives us better:

- **Organization**
- **Data protection**
- **Code reuse**

- Maintainability
- Extensibility
- Modularity

Interview answer

If interviewer asks:

"Why do we use OOP?"

A good concise answer:

OOP organizes software around objects that combine state and behavior. It helps us manage large systems through encapsulation, abstraction, inheritance and polymorphism, improving modularity, maintainability, reuse and extensibility.

3. The Four Pillars of OOP

The four major pillars are:

PillarMain idea**Encapsulation**Bundle data + methods and control access**Abstraction**Hide implementation complexity and expose essentials**Inheritance**Derive a new class from an existing class**Polymorphism**Same interface, different behavior

For now, we'll only introduce the last two and go deep into the first two.

4. Encapsulation

What is encapsulation?

Encapsulation means bundling data and the methods that operate on that data into a single unit, while controlling access to that data.

In C++, that unit is usually a **class**.

Example:

```
class BankAccount {
private:
    double balance = 0;

public:
    void deposit(double amount) {
        if (amount > 0)
            balance += amount;
    }

    double getBalance() {
        return balance;
    }
};
```

Here:

```
BankAccount
├─ balance      ← data
├─ deposit()    ← behavior
└─ getBalance() ← behavior
```

They are encapsulated inside the class.

Why do we need encapsulation?

Suppose `balance` were public:

```
class BankAccount {
public:
    double balance;
};
```

Then anyone could do:

```
account.balance = -100000;
```

That's dangerous.

Instead:

```
class BankAccount {  
private:  
    double balance;  
  
public:  
    void deposit(double amount) {  
        if (amount > 0)  
            balance += amount;  
    }  
};
```

Now the class controls **how its data can be modified**.

That's the important idea.

Encapsulation = controlled access to internal state.

5. How is encapsulation achieved?

In C++, mainly through:

Access modifiers

```
private  
protected  
public
```

For example:

```
class Student {  
private:  
    int marks;  
  
public:  
    void setMarks(int m) {  
        if (m >= 0 && m <= 100)  
            marks = m;  
    }  
  
    int getMarks() {  
        return marks;  
    }  
};
```

The outside world cannot directly modify `marks`.

Instead:

```
Outside world  
  ↓  
setMarks()  
  ↓  
validation  
  ↓  
marks
```

This gives the class control over its own state.

6. Important interview point: Encapsulation ≠ just data hiding

A common misconception:

"Encapsulation means making variables private."

That's **incomplete**.

Private members are one way of achieving encapsulation.

The broader idea is:

Keep related state and behavior together and control how the state is accessed/modified.

So don't answer:

"Encapsulation means hiding data."

That's closer to **data hiding**.

7. Real-world example of Encapsulation

Think about an ATM.

You don't directly manipulate:

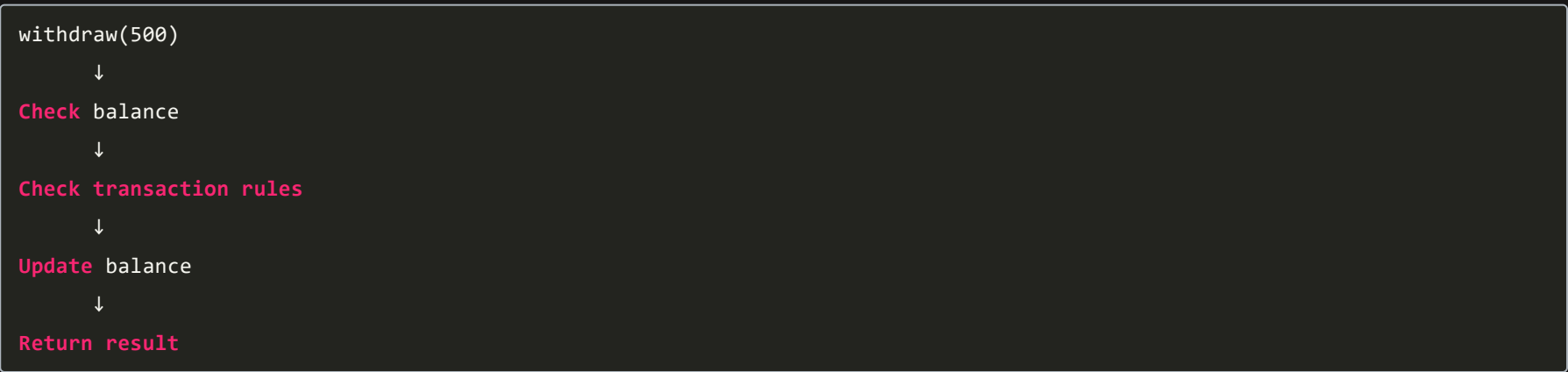
```
bank_database.balance
```

Instead, you interact through operations:

```
withdraw()  
deposit()  
checkBalance()
```

The system controls what happens internally.

For example:



You don't directly modify the balance.

That's encapsulation.

8. Abstraction

Now comes the important distinction.

What is abstraction?

Abstraction means exposing only the essential functionality while hiding unnecessary implementation details.

The key question is:

```
What should the user see?
```

rather than:

```
How does everything internally work?
```

9. Real-world example: Car

Suppose you drive a car.

You interact with:

```
steering wheel  
accelerator  
brake  
gear
```

You don't need to know:

```
fuel injection  
combustion  
crankshaft  
pistons  
ECU  
transmission internals
```

You simply use:

```
accelerate()  
brake()  
steer()
```

The complicated implementation is hidden.

That's **abstraction**.

Mental model

Abstraction = Show WHAT, hide HOW.

10. Abstraction in C++

A common way to achieve abstraction is through **abstract classes** and **pure virtual functions**.

Example:

```
class Shape {  
public:  
    virtual double area() = 0;  
};
```

This says:

Every `Shape` must provide an `area()` operation.

But `Shape` doesn't specify how the area is calculated.

Then:

```
class Circle : public Shape {  
public:  
    double area() override {  
        return 3.14 * r * r;  
    }  
  
private:  
    double r;  
};
```

And:

```
class Rectangle : public Shape {  
public:  
    double area() override {  
        return length * width;  
    }  
  
private:  
    double length, width;  
};
```

The user can simply do:

```
shape->area();
```

They don't need to know the implementation.

11. Abstraction is not limited to abstract classes

Very important interview point.

You can achieve abstraction through multiple mechanisms.

For example:

Classes

```
class Car {  
public:  
    void start();  
};
```

The user only needs:

```
car.start();
```

They don't need to know the internal implementation.

Abstract classes / interfaces

```
class Payment {
public:
    virtual void pay() = 0;
};
```

Access control

Private implementation details can also be hidden from users.

So don't say:

"Abstraction means using abstract classes."

That's too narrow.

12. Encapsulation vs Abstraction

This is **one of the most commonly asked OOP interview questions**.

EncapsulationAbstractionMain ideaBundle + control accessHide complexityFocusHow data is accessedWhat functionality is exposedQuestion"Who can access this?"What should the user know?"HidesInternal state/accessImplementation complexityCommon C++ toolsClasses, access modifiersAbstract classes, pure virtual functionsGoalProtect/manage stateReduce complexity

The easiest way to remember:

Encapsulation → DATA hiding/control

Abstraction → IMPLEMENTATION hiding

13. The ATM example makes the difference clear

Imagine an ATM.

Encapsulation

Your bank balance is protected.

You can't do:

balance = -50000

You have to use:

withdraw()
deposit()

So:

How is the data protected? → Encapsulation

Abstraction

You press:

Withdraw ₹500

You don't know:

How ATM contacts bank server
How authentication happens
How database is updated
How transaction is logged
How money dispenser works

So:

How are implementation details hidden? → Abstraction

14. Another example — Smartphone

Suppose:


```
class Smartphone {
private:
    Battery battery;

public:
    void makeCall(string number);
};
```

Encapsulation

`battery` is internal and controlled.

You cannot arbitrarily manipulate:

```
phone.battery.voltage = -100;
```

The object controls its internal state.

Abstraction

You simply call:

```
phone.makeCall("9876543210");
```

You don't care about:

```
network protocol
radio communication
SIM authentication
signal processing
packet transmission
```

That's abstraction.

15. Inheritance — Introduction

Inheritance allows one class to acquire properties and behaviors of another class.

Example:

```
class Animal {
public:
    void eat() {
        cout << "Eating";
    }
};

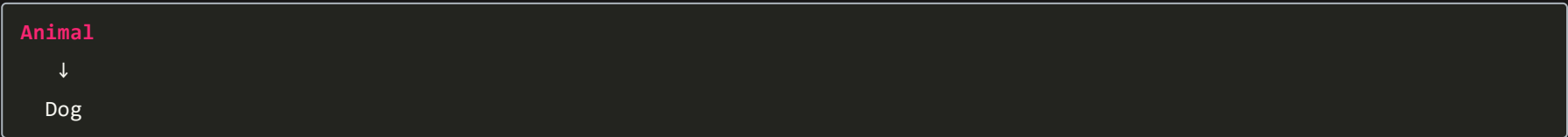
class Dog : public Animal {
};
```

Now:

```
Dog d;
d.eat();
```

`Dog` inherits from `Animal`.

Mental model:

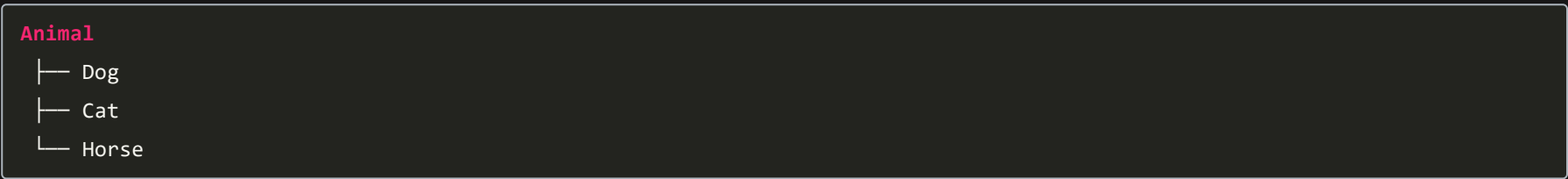


Why?

Primarily for:

- Code reuse
- Representing an **is-a** relationship
- Supporting polymorphism

Example:



We'll cover inheritance properly later.

16. Polymorphism — Introduction

Polymorphism means one interface can represent different behaviors.

Example:

```
class Animal {
public:
    virtual void sound() {
        cout << "Animal sound";
    }
};

class Dog : public Animal {
public:
    void sound() override {
        cout << "Bark";
    }
};

class Cat : public Animal {
public:
    void sound() override {
        cout << "Meow";
    }
};
```

Now:

```
Animal* animal;

animal = new Dog();
animal->sound();    // Bark

animal = new Cat();
animal->sound();    // Meow
```

Same call:

```
animal->sound();
```

Different behavior.

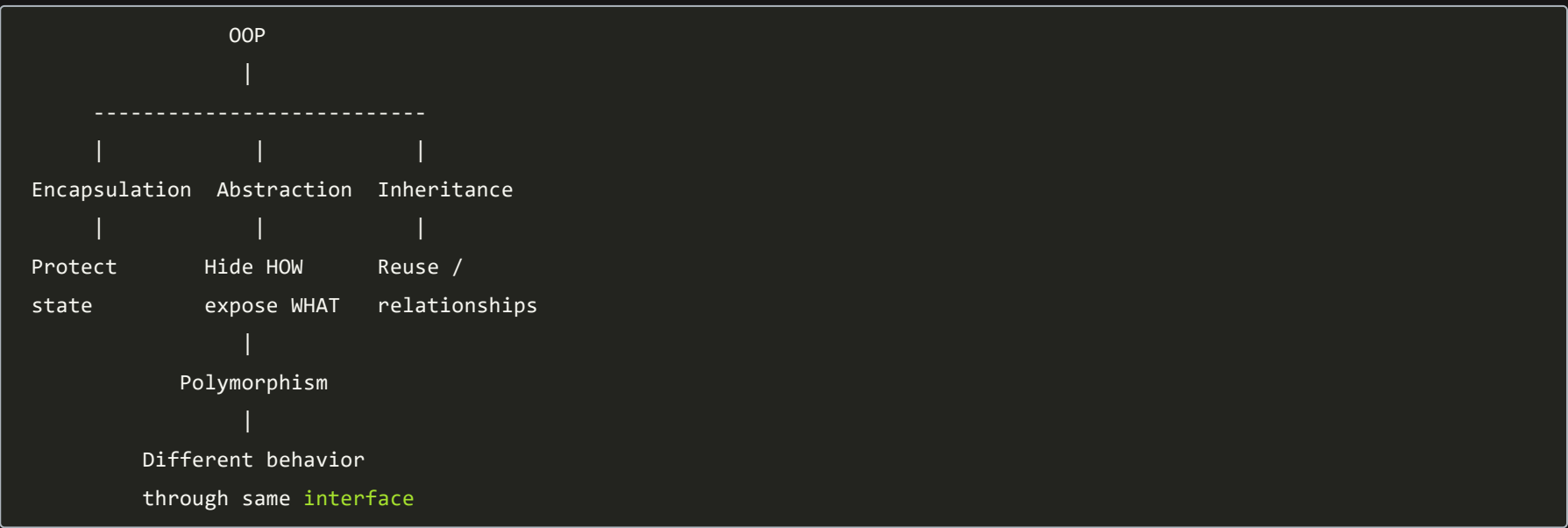
That's polymorphism.

We'll later distinguish:

- Compile-time polymorphism
- Runtime polymorphism
- Function overloading
- Operator overloading
- Function overriding
- Virtual functions

17. The Four Pillars — Mental Model

Think of a large application like this:



Or even simpler:

Encapsulation → Protect the inside

Abstraction → Hide the complexity

Inheritance → Reuse/extend

Polymorphism → Same interface, different behavior

Interview Questions You Should Know

Q1. What is OOP?

Answer:

OOP is a programming paradigm that organizes software around objects containing state and behavior. Its major principles are encapsulation, abstraction, inheritance and polymorphism.

Q2. Why is OOP useful?

Mention:

Modularity + Maintainability + Reusability + Extensibility + Data protection

Don't just say "because it makes code reusable."

Q3. What is encapsulation?

Encapsulation is bundling data and the operations that work on that data into a class while controlling access to the internal state.

Q4. What is abstraction?

Abstraction exposes the essential interface while hiding unnecessary implementation details.

Q5. Difference between abstraction and encapsulation?

The **best short answer**:

Encapsulation is about **controlling access to an object's state**, whereas abstraction is about **hiding implementation complexity and exposing only essential functionality**.

Q6. Does encapsulation mean data hiding?

Better answer:

Data hiding is an important aspect of encapsulation, but encapsulation is broader. It combines related data and behavior and controls access to the object's internal state.

Q7. Can abstraction exist without an abstract class?

Yes.

Abstraction is a concept, not a specific language feature. It can be achieved using classes, interfaces/abstract classes, access control, APIs, etc.

Q8. Is `private` enough to achieve abstraction?

Not necessarily.

`private` mainly provides **access control/data hiding**.

Abstraction is about designing an interface that exposes only what users need.

Common Interview Traps

Trap 1

"Encapsulation = data hiding."

✗ Incomplete.

Better:

Encapsulation = bundling + controlled access.

Trap 2

"Abstraction = abstract class."

✗ Too narrow.

Better:

Abstract class is one mechanism for implementing abstraction.

Trap 3

"Inheritance is always good for code reuse."

Not necessarily.

Inheritance should model a genuine **is-a** relationship.

Dog	IS-A	Animal	☑
Car	IS-A	Engine	✗

We'll later discuss why **composition is often preferred over inheritance**.

🧠 Final Cheat Sheet

ConceptRemember thisOOPOrganize code around objects**Object**State + Behavior**Encapsulation**Bundle + control access**Abstraction**Hide HOW, expose WHAT**Inheritance**IS-A relationship / extend existing class**Polymorphism**Same interface → different behavior

The most important distinction:

ENCAPSULATION
"What can I access/change?"
↓
Controls access to internal state
ABSTRACTION
"What do I need to know/use?"
↓
Hides implementation complexity

If you remember just **one sentence**:

Encapsulation protects the object's inside; abstraction simplifies what the outside world sees.